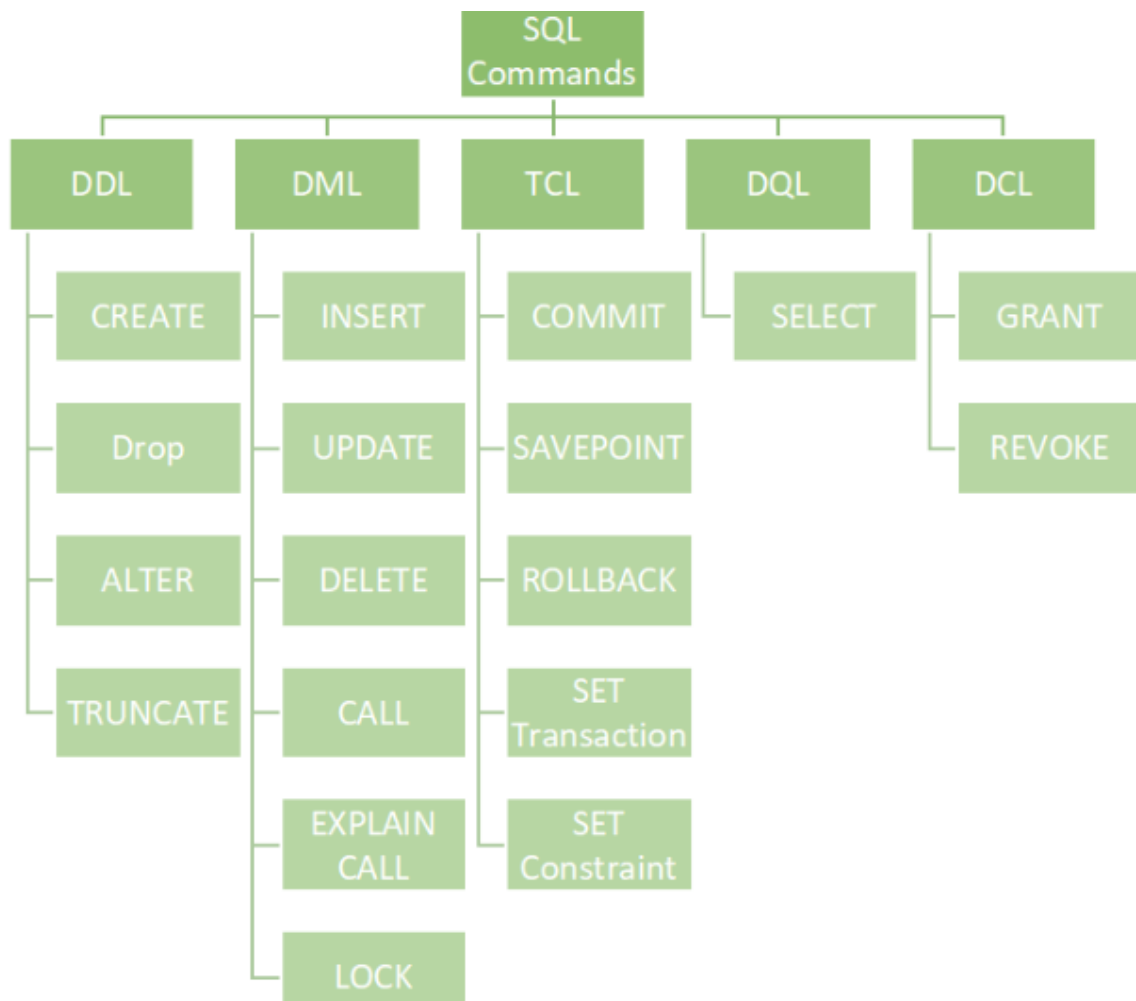




Data Definition Language (DDL)

DDL is used for [specifying the database schema](#). It is used for creating tables, schema, indexes, constraints etc. in database. Let's see the operations that we can perform on database using DDL:

- To create the database instance – **CREATE**

CREATE DATABASE databaseName;

You can verify the successful creation of database using show databases statement.

SHOW DATABASES;

- To alter the structure of database – **ALTER**

The ALTER TABLE statement is used to add, delete, or modify columns in an existing table.

To add a column in a table, use the following syntax:

`ALTER TABLE table_name ADD column_name datatype;`

To delete a column in a table, use the following syntax (notice that some database systems don't allow deleting a column):

`ALTER TABLE table_name
DROP COLUMN column_name;`

To rename a column in a table, use the following syntax:

`ALTER TABLE table_name
RENAME COLUMN old_name to new_name;`

OR

`Alter Table table_name change old_name new_name data_type;`

- To drop database instances – **DROP**

The DROP DATABASE statement is used to drop an existing SQL database.

`DROP DATABASE databasename;`

The DROP TABLE statement is used to drop an existing table in a database.

`DROP TABLE table_name;` → Completely deletes a table (structure + data)

- To delete tables in a database instance – **TRUNCATE**

The TRUNCATE TABLE statement is used to delete the data inside a table, but not the table itself.

`TRUNCATE TABLE table_name;`

- To rename database instances – **RENAME**

The **SQL RANME DATABASE...TO** statement is used to rename an existing user-created database.

`RENAME DATABASE OldDatabaseName TO
NewDatabaseName;`

- To Comment – **Comment**

Use -- to write a **comment on a single line**.

-- This query selects all students from the student table
SELECT * FROM student;

All of these commands either defines or update the database schema that's why they come under Data Definition language.

Data Manipulation Language (DML)

DML is used for accessing and manipulating data in a database. The following operations on database comes under DML:

- To read records from table(s) – **SELECT**

SELECT column1, column2, ...FROM table_name WHERE
condition;

- To insert record(s) into the table(s) – **INSERT**

INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);

- Update the data in table(s) – **UPDATE**

UPDATE table_name SET column1 = value1, column2 = value2,
...WHERE condition;

- Delete all the records from the table – **DELETE**

DELETE FROM table_name WHERE condition;

Data Control language (DCL) → (grant, revoke)

DCL is used for granting and revoking user access on a database –

- To grant access to user – **GRANT**

First, we need to create a new user. The syntax is:

CREATE USER name IDENTIFIED BY 'password';

Eg: create user 'abin' identified by 'ncs007';

Next, execute the SHOW GRANT statement to check the privileges assigned to "abin" using the following query:

SHOW GRANTS FOR abin;

GRANT privilege_name(s) ON object TO user_name;

Eg: grant all on employee to abin;

- To revoke access from user – REVOKE

REVOKE privilege_name ON object_name FROM user_name;

Eg: revoke all on employee from abin;

Transaction Control Language (TCL)→ (commit,rollback,savepoint).

The changes in the database that we made using DML commands are either performed or rolled back using TCL.

- COMMIT

The COMMIT command is used to save changes made during a transaction to the database permanently:

Eg: -- SQL statements

COMMIT;

- ROLLBACK

The ROLLBACK command is used to undo changes made during a transaction:

Eg: -- SQL statements

ROLLBACK;

- SAVEPOINT

The SAVEPOINT command allows you to set a point within a transaction to which you can later roll back:

Eg: -- SQL statements

SAVEPOINT my_savepoint;

-- More SQL statements

ROLLBACK TO my_savepoint;

Accessing database (SELECT, Filtering using WHERE, HAVING, GROUP BY, ORDER BY Clauses)

The GROUP BY clause is used to **group records** based on one or more columns.

The HAVING clause in SQL is similar to the WHERE clause that is used to filter the data but in a different way. HAVING clause is used to filter the result obtained by the GROUP BY clause. It is used with the aggregation function. It can include one or more conditions

The ORDER BY clause is used to **sort data** in **ascending (ASC) or descending (DESC) order**.

Syntax

SELECT column_names

FROM table_name

WHERE conditions

GROUP BY column_name

HAVING conditions

ORDER BY column_name;

Creating Views

- Views in SQL are considered as a virtual table. A view also contains rows and columns.

- To create the view, we can select the fields from one or more tables present in the database.

Database views are created using the **CREATE VIEW** statement. Views can be created from a single table, multiple tables or another view.

The basic **CREATE VIEW** syntax is as follows –

```
CREATE VIEW view_name AS
SELECT column1, column2.....
FROM table_name
WHERE [condition];
```

Subquery

A subquery in MySQL is a query, which is nested into another SQL query and embedded with SELECT, INSERT, UPDATE or DELETE statement along with the various operators. We can also nest the subquery with another subquery. A subquery is known as the **inner query**, and the query that contains subquery is known as the **outer query**.

```
SELECT column_list (s) FROM table_name
WHERE column_name OPERATOR
(SELECT column_list (s) FROM table_name [WHERE])
```

PL/SQL (Procedural Language/SQL) programs are:

Blocks of code that combine **SQL statements** with **programming features** like conditions, loops, and variables.

PL/SQL

PL/SQL, which stands for Procedural Language extensions to the **Structured Query Language** (SQL). It is a combination of SQL along with the procedural features of programming languages.

SQL Trigger

A trigger is a stored procedure in a database that automatically invokes whenever a special event in the database occurs. For example, a trigger can be

invoked when a row is inserted into a specified table or when specific table columns are updated in simple words a trigger is a collection of SQL statements with particular names that are stored in system memory.

Syntax:

```
create trigger [trigger_name]  
[before | after]  
{insert | update | delete}  
on [table_name]  
[for each row]  
[trigger_body]
```

1. **Create trigger [trigger_name]:** Creates or replaces an existing trigger with the trigger_name.
2. **[before | after]:** This specifies when the trigger will be executed.
3. **{insert | update | delete}:** This specifies the DML operation.
4. **On [table_name]:** This specifies the name of the table associated with the trigger.
5. **[for each row]:** This specifies a row-level trigger, i.e., the trigger will be executed for each affected row.
6. **[trigger_body]:** This provides the operation to be performed as the trigger is fired

Execution of cursor

The major function of a cursor is to retrieve data, one row at a time, from a result set, unlike the SQL commands which operate on all the rows in the result set at one time. Cursors are used when the user needs to update records in a singleton fashion or in a row by row manner, in a database table.

Cursor Actions

- **Declare Cursor:** A cursor is declared by defining the SQL statement that returns a result set.
- **Open:** A Cursor is opened and populated by executing the SQL statement defined by the cursor.
- **Fetch:** When the cursor is opened, rows can be fetched from the cursor one by one or in a block to perform data manipulation.
- **Close:** After data manipulation, close the cursor explicitly.

What is MongoDB?

MongoDB, the most popular NoSQL database, is an open-source document-oriented database. The term 'NoSQL' means 'non-relational'. It means that MongoDB isn't based on the table-like relational database structure but provides an altogether different mechanism for storage and retrieval of data. Unlike standard relational databases, MongoDB stores data in a JSON document structure form. This makes it easy to operate with dynamic and unstructured data and MongoDB is an open-source and cross-platform database System.

Create a Database

You can change or create a new database by typing `use` then the name of the database.

1) Eg: `use test`

Create Collection same as table in SQL)

You can create a collection using the `createCollection()` database method.

Eg: `db.createCollection("student")`

Insert Documents (Documents are equivalent to records or rows in a relational database table)

```
db.student.insert({rollno:101,name:"Alex",Branch:"MCA"})
```

```
db.student.insert({rollno:102,name:"Maya",Branch:"MBA"})
```

```
db.student.insert({rollno:101,name:"Niya",Branch:"BCA"})
```

To select data from a collection in MongoDB, we can use the `find()` method.

```
db.student.find().pretty() // to display table in neat format
```